CSE110A: Compilers

Topics:

- *Module 4 – BASIC BLOCKS and CFGS*
  - *Basic Blocks*
  - *Control Flow Graphs*

Optimization categories

- Machine-independent - these optimizations should work well across many different systems
  - Examples?
  - All the examples we looked at before seem like they will help across many systems
- Machine dependent - these optimizations start to optimize the code for a given system
  - Examples?
  - loop chunking for cache line size and vectorization.
  - instruction re-orderings to take advantage of processor pipelines.
  - fused multiply-and-add instructions
  - vector processing (parallel computation)

Optimization categories

- **Machine-independent** - these optimizations should work well across many different systems
  - Examples?
  - All the examples we looked at before seem like they will help across many systems
- In this module we will be looking at machine-independent optimizations.
- What are the pros of machine-independent optimizations?

Optimization categories

Next category level is how much code we need to reason about for the optimization.

- **Local optimizations**: examine a ”basic block”, i.e. a small region of code with no control flow.
  - Examples?
- **Regional optimizations**: several basic blocks with simple control flow.
  - Examples?
- **Global optimization:** optimizes across an entire function

Optimization categories

- **Local optimizations**: examine a ”basic block”, i.e. a small region of code with no control flow.
- **Regional optimizations**: several basic blocks with simple control flow
- **Global optimization:** optimizes across an entire function
- **IDFA Inter-Procedural Data Flow Analysis** : Optimizes across functions

Discussion:

- What are the pros and cons of each?
- Going across functions is less common, why?

Optimization categories

- **Local optimizations**: examine a ”basic block”, i.e. a small region of code with no control flow.
- **Regional optimizations**: several basic blocks with simple control flow
- **Global optimization:** optimizes across an entire function

For this module:

- We will look at two optimizations in detail:
- A local optimization: Local value numbering
- A regional optimization: Loop unrolling
- We will implement both as homework
- We will discuss several other optimizations and analysis

BASIC BLOCKS

IR Program structure

- A sequence of 3 address instructions

- Programs can be split into **Basic Blocks:**
  - A **sequence of 3 address instructions** such that:
    there is **a single entry, single exit**

- *Important property*: an instruction in a basic block can assume that all preceding instructions will execute

Single Basic Block

```
Label_x:
op1;
op2;
op3;
br label_z;
```

IR Program structure

- A sequence of 3 address instructions
- Programs can be split into **Basic Blocks:**
  - A **sequence of 3 address instructions** such that: there is **a single entry, single exit**
- *Important property*: an instruction in a basic block can assume that all preceding instructions will execute

Single Basic Block

```
Label_x:
op1;
op2;
op3;
br label_z;
```

Two Basic Blocks

```
Label_x:
op1;
op2;
op3;

Label_y:
op4;
op5;
```

IR Program structure

How might they appear in a high-level language? What are some examples?

- A sequence of 3 address instructions
- Programs can be split into **Basic Blocks:**
  - A **sequence of 3 address instructions** such that: there is **a single entry, single exit**
- *Important property*: an instruction in a basic block can assume that all preceding instructions will execute

Single Basic Block

```
Label_x:
op1;
op2;
op3;
br label_z;
```

Two Basic Blocks

```
Label_x:
op1;
op2;
op3;

Label_y:
op4;
op5;
```

IR Program structure

- A sequence of 3 address instructions
- Programs can be split into **Basic Blocks:**
  - A **sequence of 3 address instructions** such that: there is **a single entry, single exit**
- *Important property*: an instruction in a basic block can assume that all preceding instructions will execute

How might they appear in a high-level language?

How many basic blocks?

```
…
if (x) {
    …
}
else {
   …
}
…
```

Two Basic Blocks

```
Label_x:
op1;
op2;
op3;

Label_y:
op4;
op5;
```

Single Basic Block

```
Label_x:
op1;
op2;
op3;
br label_z;
```

Converting 3 address code into basic blocks

- Let’s try an example: test 4 in HW 4:

Converting 3 address code into basic blocks

- Simple algorithm:
  - keep a list of basic blocks
  - a basic block is a list of instructions

  - Iterate over the 3 address instructions
  - if you see a branch or a label, finalize the current basic block and start a new one.

Converting 3 address code into basic blocks

```
basic_blocks = []
bb = []

for instr in program:
    # If label starts a new block
    if instr.type == "label":
        if bb:                  # close previous block
            basic_blocks.append(bb)
        bb = [instr]            # start new block
    else:
        bb.append(instr)
        # If branch ends a block
        if instr.type == "branch":
            basic_blocks.append(bb)
            bb = []

# append final block if non-empty
if bb:
    basic_blocks.append(bb)
```

Optimization levels

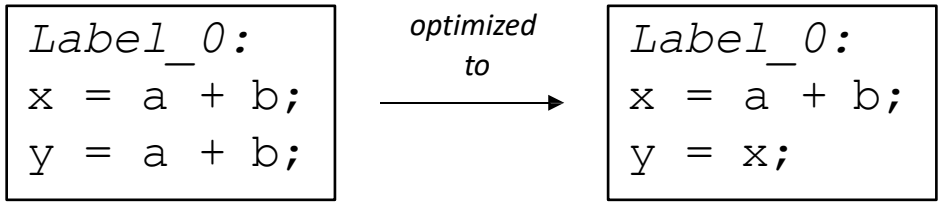
- **Local optimizations:**
  - Optimizes an individual basic block
- **Regional optimizations:**
  - Combines several basic blocks
- **Global optimizations:**
  - operates across an entire procedure
  - what about across procedures?

Optimization levels

```
Label_0:
x = a + b;
y = a + b;
```

- **Local optimizations:**
  - Optimizes an individual basic block
- **Regional optimizations:**
  - Combines several basic blocks
- **Global optimizations:**
  - operates across an entire procedure
  - what about across procedures?

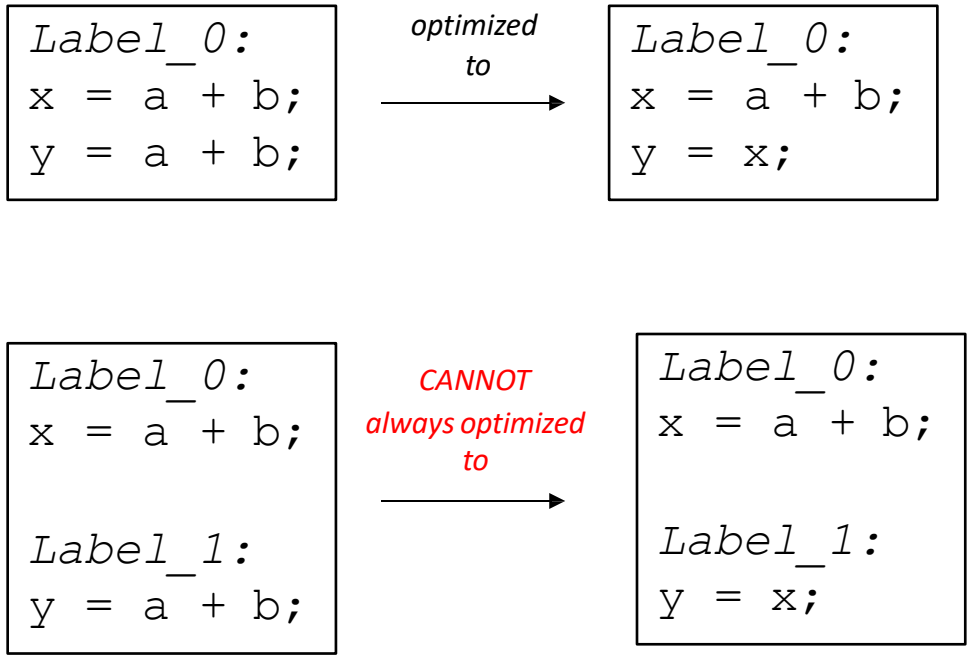
Optimization levels



- **Local optimizations:**
  - Optimizes an individual basic block
- **Regional optimizations:**
  - Combines several basic blocks
- **Global optimizations:**
  - operates across an entire procedure
  - what about across procedures?

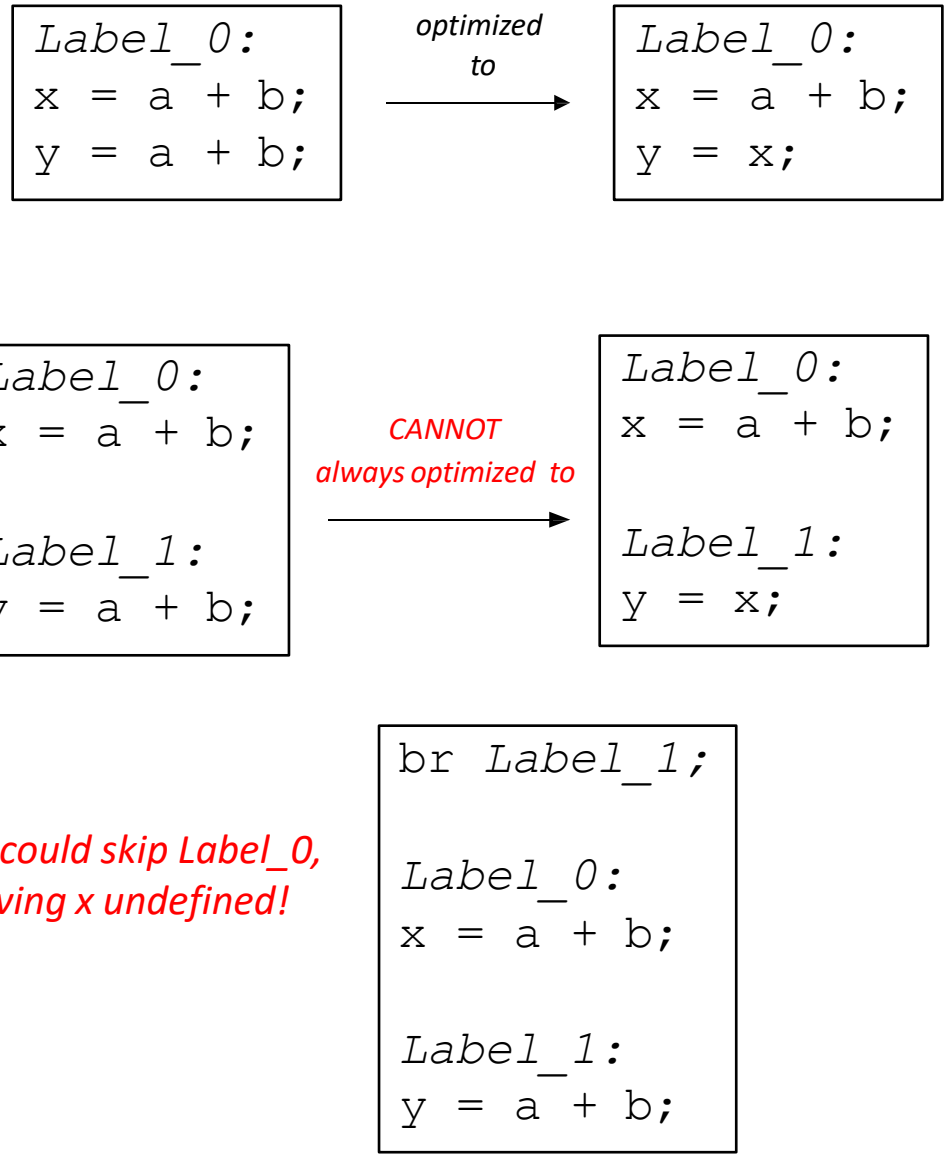
Optimization levels

- **Local optimizations:**
  - Optimizes an individual basic block
- **Regional optimizations:**
  - Combines several basic blocks
- **Global optimizations:**
  - operates across an entire procedure
  - what about across procedures?



Optimization levels

- **Local optimizations:**
  - Optimizes an individual basic block
- **Regional optimizations:**
  - Combines several basic blocks
- **Global optimizations:**
  - operates across an entire procedure
  - what about across procedures?



Regional Optimization

```
…
if (x) {
    …
}
else {
  x = a + b;
}
y = a + b;
…
```

we cannot replace:
y = a + b.
with
y = x;

Regional Optimization: Data Flow Analysis

```
…
if (x) {
    …
}
else {
  x = a + b;
}
y = a + b;
…
```

we cannot replace:
y = a + b.
with
y = x;

```
x = a + b;
if (x) {
    …
}
else {
  …
}
y = a + b;
…
```

Can x = a + b be hoisted out of the if then else?

In that case, we can check if a and b are not redefined, then
y = a + b;
can be replaced with
y = x;

This requires regional analysis and optimizations

Optimizing over wider regions

- Local value numbering operates over just one basic block.
- We want optimizations that operate over several basic blocks (a region), or across an entire procedure (global)
- For this, we need Control Flow Graphs and Flow Analysis
  - We may have time to discuss this later in the module

Control Flow Graphs (CFG)

Understanding Program Execution Flow

What is a CFG?

- A Control Flow Graph (CFG) is a representation of all paths that might be traversed through a program during its execution.
- Nodes → represent basic blocks (a sequence of instructions with no branches except at the end).
- Edges → represent control flow between these blocks.
- Purpose: Helps in compiler optimizations, program analysis, and detecting unreachable code.

Components of a CFG

1. Basic Block: Sequence of statements with a single entry and exit point.
2. Nodes: Represent basic blocks.
3. Edges: Show the flow of control between blocks.
4. Entry/Exit Nodes: Designate the start and end of the CFG.

Example Program

```
int main() {
    int x = 0;
    if (x > 0) {
        x = x + 1;
    } else {
        x = x – 1;
    }
    return x;
}
```

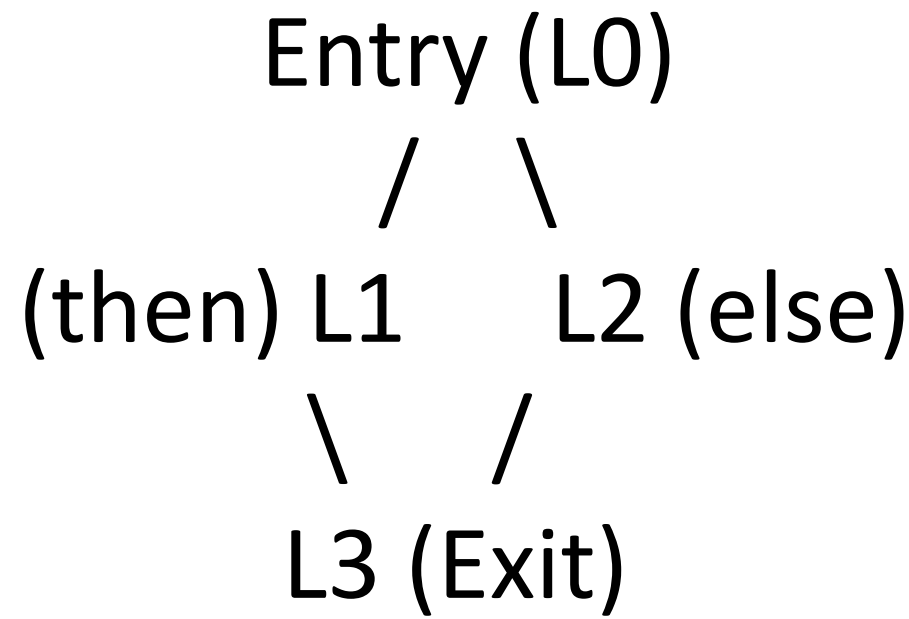
Example Program as 3-address code

```
L0:                 // Main entry point
t1 = 0              // x = 0
x = t1
if x > 0
    goto L1         // then branch
else
    goto L2         // else branch
L1:                 // then block
   t2 = x + 1
   x = t2
   goto L3          // goto end of if-else statement
L2:                 // else block
   t3 = x - 1
   x = t3
   goto L3          // goto end of if-else statement

L3:                 // after if-else
   return x
```

Each label here represents a basic block

Example Program as 3-address code



Why CFG is Important

Compiler Optimizations: Dead code elimination, loop optimization, instruction scheduling.
Program Analysis: Detect unreachable code, infinite loops, or security vulnerabilities.
Testing & Verification: Path coverage for testing and formal verification.

Types of CFG Edges

Conditional Edges:

Result from if/else statements or loops.

Unconditional Edges:

Straight-line flow between sequential blocks.

Back Edges:

Point back to earlier blocks (used in loops).

Summary

CFG is a graph-based representation of program execution.

Nodes = basic blocks; Edges = control flow.

Useful in compiler design, testing, optimizations
and program analysis.

Enables better understanding of complex program structures.